

Trường Đại học Công nghệ Thông tin

Đại học Quốc gia Tp.Hồ Chí Minh

Toàn diện – Sáng tạo - Phụng sự



NỘP BÀI TẬP THỰC HÀNH BUỔI 3

Môn học: Kỹ thuật xây dựng hệ thống Agentic AI (SE373)

Bài tập: Dựn agent đặt vé bằng LangChain

Họ và tên sinh viên: Trần Minh Thuận

Mã số sinh viên: 24521746

1) THIẾT KẾ CÁC LỚP HARNESS BẢO VỆ

- Trong kiến trúc Agentic Engineering, hệ thống tuân thủ công thức:

$$\text{Agent} = \text{Goal} + \text{Tools} + \text{Loop} + \text{Termination}$$

- Harness đóng vai trò là khung bao bọc bằng code cố định để kiểm soát hành vi, chi phí và sự an toàn của mô hình. **Hệ thống đã cài đặt đầy đủ 4 lớp Harness tiêu chuẩn:**

❖ Lớp 1: Ràng buộc là dữ liệu (`constraints.py`)

- **Mục đích:** Thay vì để yêu cầu của người dùng phân tán dưới dạng chuỗi câu lệnh prompt tự do khiến mô hình dễ bị trôi mục tiêu, toàn bộ tiêu chí đặt vé được chuẩn hóa thành lớp đối tượng dữ liệu Pydantic `BookingConstraint`.
- **Cấu trúc:** Quản lý chặt chẽ các trường: điểm khởi hành (`origin`), điểm đến (`dest`), ngày bay (`date`), trần ngân sách (`max_price`), giờ bay (`depart_before`), tên hành khách (`passenger_name`), hành lý ký gửi (`baggage_kg`) và chính sách hoàn hủy (`require_refundable`).
- **Tác dụng:** Cung cấp phương thức `to_natural_language()` làm ngữ cảnh định hướng cho Model, đồng thời đóng vai trò làm hệ quy chiếu bất biến (`Ground Truth`) để Harness Sensor kiểm định độc lập ở khâu cuối cùng.

❖ Lớp 2: Kiểm quyền trước khi gọi tool (`permission.py`)

- **Vị trí can thiệp:** Thực hiện chặn và kiểm tra (`Interception Check`) trước khi bắt kỳ công cụ nhạy cảm nào được kích hoạt.
- **Logic can thiệp:**
 - Bắt chặn công cụ `book_ticket` để yêu cầu con người phê duyệt trực tiếp qua cơ chế `Human-in-the-loop CLI` (`[y/n]`).
 - Tự động phát cảnh báo và ngắt thao tác nếu phát hiện loại vé không hoàn tiền (`refundable == False`).
 - Bắt chặn nếu phát hiện tổng chi phí (giá vé cơ bản + cước hành lý) vượt trần ngân sách cho phép.
- **Tác dụng:** Ngăn chặn tuyệt đối các rủi ro tổn thất tài chính ngoài ý muốn do Model tự ý quyết định. Đồng thời hỗ trợ cờ `auto_approve` cho các kịch bản chạy benchmark tự động.

❖ Lớp 3: Tiêu chí hoàn thành kiểm bằng code (`completion_sensor.py`)

- **Bản chất:** Áp dụng `Sensor Computational` thay vì dùng `Sensor Inferential`.

- **Logic thẩm định:** Hợp lệ $\Leftrightarrow$ (Mã vé $\in$ MockDB) $\wedge$ (Status == "confirmed") $\wedge$ (Paid == True) $\wedge$ (Tổng giá $\leq$ Ngân sách) $\wedge$ (Đúng ngày, giờ, hành lý)

- **Tác dụng:** Loại trừ triệt để hiện tượng Model tự ảo giác xác nhận đã đặt vé thành công trong khi hệ thống backend chưa hề ghi nhận dữ liệu.

❖ **Lớp 4: Cơ chế bàn giao cho con người (handoff.py) & Phát hiện lặp (loop_detector.py)**

- **Bộ phát hiện lặp (LoopDetector):** Lưu trữ dấu vân tay (tool_name, args_hash) trong hàng đợi trượt. Nếu cùng một hành vi lặp lại quá ngưỡng quy định mà không tạo ra tiến triển mới, hệ thống sẽ ngắt luồng ngay lập tức để tiết kiệm chi phí token.
- **Báo cáo bàn giao (HandoffReport):** Khi bị ngắt do chạm quyền kiểm soát, phát hiện lặp hoặc kế hoạch bị gián đoạn, hệ thống xuất báo cáo có cấu trúc chuẩn Pydantic gồm: Trạng thái hiện tại, Các bước đã thử, Hành động gây lỗi và Câu hỏi hành động ngắn gọn giúp con người đưa ra quyết định xử lý chỉ trong 30 giây.

```
D:\SE373\flight_booking_agent\.venv\Scripts\python.exe D:\SE373\flight_booking_agent\test_manual.py
=====
BẮT ĐẦU KIỂM THỬ ĐỘC LẬP TỪNG LỚP HARNESS
=====

[1] Lớp Ràng buộc dữ liệu (Constraints):
    -> Đã khởi tạo mục tiêu dạng Pydantic: {'origin': 'SGN', 'dest': 'DAD', 'date': '2026-10-07', 'max_price': 2200000, 'depart_before': '12:00', 'passenger_name': 'Nguyen Van A', 'bags': 1}

[2] Lớp Kiểm quyền (Permission Guard):
    - Thử gọi 'pay': Cho phép = False | Lý do: Hành động thanh toán tái chính (pay) là nhạy cảm, bắt buộc con người phê duyệt.
    - Thử đặt vé non-refundable: Cho phép = False | Lý do: CẢNH BÁO: Chuyến bay 'VN122' là vé KHÔNG HOÀN TIẾN (non-refundable). Cần xác nhận.
    - Thử đặt vé khi auto_approve=True: Cho phép = True

[3] Bộ phát hiện Lặp (Loop Detector):
    - Vòng 1 gọi 'search_flights': None
    - Vòng 2 gọi lại đúng tham số: LOOP

[4] Tiêu chí hoàn thành (Completion Sensor):
    - Kiểm tra mã vé rác: Thành công = False | Chi tiết: Mã đặt chỗ 'BK_KHONG_TON_TAI' không tồn tại trong hệ thống (Mock DB).
    - Kiểm tra vé QH118 đã đặt: Thành công = True | Chi tiết: XÁC MINH HỢP LỆ (100% PASS): Đã xác nhận vé BK_QH118_05D cho Nguyen Van A, chuyến QH118 ngày 2026-10-07 lúc 10:00, tổng giá 2.200.000 VND.

[5] Lớp Bàn giao (Handoff Report):
    - Định dạng xuất terminal hợp lệ:
```

```
=====
BÁO CÁO BÀN GIAO CHO CON NGƯỜI (HUMAN HANDOFF REPORT)
=====
[Trạng thái]      : KIỂM THỬ BÀN GIAO
[Giai đoạn đúng]  : Kiểm thử đơn vị
[Số bước đã thử]  : 1 bước
[Lý do can thiệp] : Đã thử chuyển bay nhưng không khớp
[Yêu cầu cụ thể]  : Bạn có muốn tăng ngân sách để tiếp tục tìm kiếm không?
=====

>>> TẤT CẢ CÁC LỚP BẢO VỆ ĐÃ VƯỢT QUA KIỂM THỬ (100% PASS) <<<
=====
```

Hình 1: Kết quả kiểm thử độc lập 4 lớp Harness bảo vệ đạt tỷ lệ vượt qua 100% (test_manual.py)

2) CÀI ĐẶT 3 MẪU THIẾT KẾ AGENT

Cả 3 mẫu thiết kế đều được xây dựng trên thư viện LangChain, hỗ trợ gọi Model thực tế (Google Gemini / OpenAI) qua biến môi trường hoặc chạy qua bộ suy luận tích hợp:

❖ Mẫu 1: ReAct Agent (react_agent.py)

- **Nguyên lý:** Vận hành tuần tự theo chu trình phản xạ: Thought -> Action -> Observation.
- **Cơ chế:** Model tự suy luận mục tiêu tiếp theo dựa trên dữ liệu thu thập được từ bước trước. Toàn bộ tool calls được bao bọc bởi LoopDetector và PermissionGuard để đảm bảo an toàn trong từng nhịp suy luận.
- **Đặc điểm:** Tốc độ thích ứng cao, phản xạ linh hoạt với các biến động của môi trường tìm kiếm chuyến bay.

❖ Mẫu 2: Plan-then-Execute Agent (plan_execute_agent.py)

- **Nguyên lý:** Phân định rõ ràng trách nhiệm giữa khâu lập kế hoạch và thực thi:
 - i. LLM Planner: Phân tích ràng buộc và sinh ra kế hoạch thực thi có cấu trúc Pydantic JSON (ExecutionPlan) gồm các bước tuần tự.
 - ii. LLM Step Executor: Đọc ngữ cảnh của các bước trước đó để quyết định bộ tham số chính xác nhất cho từng bước công cụ.
 - iii. LLM Dynamic Re-planner: Khi một bước trong kế hoạch gặp sự cố, Re-planner sẽ nhận lại toàn bộ trace lịch sử để tái lập kế hoạch cho các bước còn lại mà không làm sụp đổ toàn bộ luồng xử lý.
- **Đặc điểm:** Tính minh bạch cao, cấu trúc các bước rõ ràng trước khi hành động.

❖ Mẫu 3: Mẫu Lai - Hybrid (hybrid_agent.py)

- **Nguyên lý:** Áp dụng kiến trúc phân tầng 2 cấp :
 - **Tầng vĩ mô :** Phân chia mục tiêu tổng thành 3 chặng chiến lược rõ ràng:
 - **Chặng 1:** Khám phá và lọc danh sách chuyến bay.
 - **Chặng 2:** Thẩm định ghế trống và tính phụ phí hành lý.
 - **Chặng 3:** Phê duyệt quyền và chốt vé chính thức.
 - **Tầng vi mô :** Ở mỗi chặng, một ReAct Sub-Agent độc lập đảm nhiệm giải quyết mục tiêu, có khả năng tự sửa lỗi cục bộ trước khi bàn giao trạng thái hoàn tất sang chặng tiếp theo.

- **Đặc điểm:** Khả năng cô lập lỗi tối đa, bảo vệ cửa sổ ngữ cảnh (Context Window) không bị quá tải.

```
HỆ THỐNG ĐẶT VÉ MÁY BAY THÔNG MINH - AGENTIC AI (SE373)
      (Agent = Model + Harness)

=====

[CHỌN MẪU THIẾT KẾ AGENT]:
1. ReAct Agent (Thought -> Action -> Observation Loop)
2. Plan-then-Execute (LLM Planner -> Step Executor -> Re-planner)
3. Hybrid (Lai - High-level Coordinator + Micro ReAct Sub-agents)

Nhập lựa chọn của bạn (1/2/3, mặc định 1): 1

--- NHẬP THÔNG TIN RÀNG BUỘC ĐẶT VÉ ---
Điểm khởi hành (VD: SGN): SGN
Điểm đến (VD: DAD): DAD
Ngày bay (VD: 2026-10-07): 2026-10-07
Ngân sách tối đa (VND, VD: 2200000): 2200000
Hành lý ký gửi bổ sung (kg, VD: 15): 15
Tên hành khách (VD: Tran Minh Thuan): Tran Minh Thuan

-----

Thông số đã thiết lập:
Điểm khởi hành: SGN
Điểm đến: DAD
Ngày bay: 2026-10-07
Thời gian khởi hành: trước 12:00
Ngân sách tối đa: 2,200,000 VND
Hành khách: Tran Minh Thuan
Hành lý ký gửi: 15 kg
```

```
-----

>>> Đang khởi chạy: ReAct Agent...

[KIỂM QUYỀN]: XÁC NHẬN ĐẶT VÉ: Chuẩn bị chốt vé chuyến QH118 (Ghế 05D) cho hành khách Nguyen Van A.
Bạn có đồng ý thực thi không? (y/n): y

=====

KẾT QUẢ THỰC THI: SUCCESS
Tổng số vòng tương tác (Turns): 4
=====

CHÚC MỪNG! ĐÃ ĐẶT VÉ THÀNH CÔNG!
Mã đặt chỗ (Booking Code): BK_QH118_05D
Chi tiết kiểm chứng độc lập: XÁC MINH HỢP LỆ (100% PASS): Đã xác nhận vé BK_QH118_05D cho Nguyen Van A, chuyến QH118 ngày 2026-10-07 lúc 10:00, tổng tiền 2,080,000 VND.

--- NHẬT KÝ BƯỚC CHẠY (EXECUTION TRACE) ---
Bước 1 [Thought]: Tôi cần tìm kiếm các chuyến bay từ SGN đến DAD vào ngày 2026-10-07....
Bước 1 [Action]: search_flights -> Args: {'origin': 'SGN', 'dest': 'DAD', 'date': '2026-10-07'}
Bước 2 [Thought]: Chuyến bay QH118 có giờ bay 10:00 và giá tốt. Tôi sẽ kiểm tra ghế trống....
Bước 2 [Action]: check_seat_availability -> Args: {'flight_id': 'QH118'}
Bước 3 [Thought]: Chuyến QH118 còn các ghế ['05D', '05E']. Khách yêu cầu 15kg hành lý, tôi tính phụ phí....
Bước 3 [Action]: calculate_baggage_fee -> Args: {'flight_id': 'QH118', 'baggage_kg': 15}
Bước 4 [Thought]: Mọi điều kiện đều đạt: Chuyến QH118, ghế 05D, chi phí hợp lệ. Tiến hành chốt đặt vé máy bay....
Bước 4 [Action]: book_ticket -> Args: {'flight_id': 'QH118', 'seat_number': '05D', 'passenger_name': 'Nguyen Van A', 'baggage_kg': 15}

Process finished with exit code 0
|
```

```
[KIỂM QUYỀN]: XÁC NHẬN ĐẶT VÉ: Chuẩn bị chốt vé chuyến QH118 (Ghế 05D) cho hành khách Nguyen Van A.
Bạn có đồng ý thực thi không? (y/n): n

=====
KẾT QUẢ THỰC THI: REJECTED_BY_USER
Tổng số vòng tương tác (Turns): 4
=====

=====
BÁO CÁO BÀN GIAO CHO CON NGƯỜI (HUMAN HANDOFF REPORT)
=====

[Trạng thái]      : TỪ CHỐI BỞI NGƯỜI DÙNG (REJECTED)
[Giải đoạn dừng]  : Vòng 4 trước khi gọi 'book_ticket'
[Số bước đã thử]  : 3 bước
[Lý do can thiệp] : Người dùng từ chối cấp quyền: XÁC NHẬN ĐẶT VÉ: Chuẩn bị chốt vé chuyến QH118 (Ghế 05D) cho hành khách Nguyen Van A.
[Yêu cầu cụ thể]  : Hành động đã bị hủy an toàn theo ý muốn người dùng.
=====

--- NHẬT KÝ BƯỚC CHẠY (EXECUTION TRACE) ---
Bước 1 [Thought]: Tôi cần tìm kiếm các chuyến bay từ SGN đến DAD vào ngày 2026-10-07...
Bước 1 [Action ]: search_flights -> Args: {'origin': 'SGN', 'dest': 'DAD', 'date': '2026-10-07'}
Bước 2 [Thought]: Chuyến bay QH118 có giờ bay 10:00 và giá tốt. Tôi sẽ kiểm tra ghế trống...
Bước 2 [Action ]: check_seat_availability -> Args: {'flight_id': 'QH118'}
Bước 3 [Thought]: Chuyến QH118 còn các ghế ['05D', '05E']. Khách yêu cầu 15kg hành lý, tôi tính phụ phí....
Bước 3 [Action ]: calculate_baggage_fee -> Args: {'flight_id': 'QH118', 'baggage_kg': 15}
```

Hình 2: Cơ chế can thiệp kiểm quyền tương tác người dùng khi chạy main.py

3) ĐÁNH GIÁ THỰC NGHIỆM & PHÂN TÍCH KẾT QUẢ

❖ Bảng kết quả thực nghiệm thu được từ hệ thống:

Kịch bản kiểm thử	Mẫu thiết kế Agent	Trạng thái thực thi	Số vòng g	Thời gian	Mã đặt chỗ	Kiểm chứng Harness
Kịch bản 1: Môi trường thuận lợi (Happy Path)	ReAct Agent	SUCCESS	4	0.02s	BK_QH118_05D	Đạt (Pass)
	Plan-then-Execute	SUCCESS	4	0.00s	BK_QH118_05D	Đạt (Pass)
	Hybrid (Lai)	SUCCESS	7	0.00s	BK_QH118_05D	Đạt (Pass)
Kịch bản 2: Biến động ghế (Dynamic Environment)	ReAct Agent	SUCCESS	5	0.00s	BK_VN122_12A	Đạt (Pass)
	Plan-then-Execute	SUCCESS	5	0.00s	BK_VN122_12A	Đạt (Pass)
	Hybrid (Lai)	SUCCESS	7	0.00s	BK_VN122_12A	Đạt (Pass)
Kịch bản 3: Kích hoạt Kiểm quyền Harness	ReAct Agent	WAIT_APPROVAL	4	0.00s	-	Chặn an toàn
	Plan-then-Execute	WAIT_APPROVAL	4	0.00s	-	Chặn an toàn

	Hybrid (Lai)	WAIT_APPROVAL	6	0.00s	-	Chặn an toàn
--	--------------	---------------	---	-------	---	--------------

```
=====
BẮT ĐẦU CHƯƠNG TRÌNH ĐÁNH GIÁ THỰC NGHIỆM 3 MẪU THIẾT KẾ AGENT (SE373)
=====

>>> Đang chạy Kịch bản 1: Môi trường thuận lợi (Happy Path)...
    Mô tả: Đặt vé SGN-DAD, kèm 15kg hành lý, ngân sách 2.3tr. Mọi chuyến đều còn ghế.

>>> Đang chạy Kịch bản 2: Biến động ghế (Dynamic Environment)...
    Mô tả: Chuyển giá tốt QH118 bị hết ghế đột ngột, Agent phải thích ứng chọn chuyến VN122.

>>> Đang chạy Kịch bản 3: Kích hoạt Kiểm quyền Harness (Permission Guard)...
    Mô tả: Chạy với auto_approve=False để kiểm tra bắt chặn hành động đặt vé nhạy cảm.


=====
BẢNG TỔNG HỢP SO SÁNH THỰC NGHIỆM (BENCHMARK REPORT)
=====
+-----+-----+-----+-----+-----+-----+-----+
| Kịch bản kiểm thử | Mẫu thiết kế Agent | Trạng thái thực thi | Số vòng | Thời gian | Mã đặt chỗ | Kiểm chứng Harness |
+-----+-----+-----+-----+-----+-----+-----+
| Kịch bản 1: Môi trường thuận lợi (Happy Path) | ReAct Agent | SUCCESS | 4 | 0.02s | BK_QH118_05D | ĐẠT (PASS) |
+-----+-----+-----+-----+-----+-----+
| | Plan-then-Execute | SUCCESS | 4 | 0.00s | BK_QH118_05D | ĐẠT (PASS) |
+-----+-----+-----+-----+-----+-----+
| | Hybrid (Lai) | SUCCESS | 7 | 0.00s | BK_QH118_05D | ĐẠT (PASS) |
+-----+-----+-----+-----+-----+-----+
| ----- | ----- | ----- | ----- | ----- | ----- | ----- |
+-----+-----+-----+-----+-----+-----+
| Kịch bản 2: Biến động ghế (Dynamic Environment) | ReAct Agent | SUCCESS | 5 | 0.00s | BK_VN122_12A | ĐẠT (PASS) |
+-----+-----+-----+-----+-----+-----+
| | Plan-then-Execute | SUCCESS | 5 | 0.00s | BK_VN122_12A | ĐẠT (PASS) |
+-----+-----+-----+-----+-----+-----+
| | Hybrid (Lai) | SUCCESS | 7 | 0.00s | BK_VN122_12A | ĐẠT (PASS) |
+-----+-----+-----+-----+-----+-----+
| ----- | ----- | ----- | ----- | ----- | ----- | ----- |
+-----+-----+-----+-----+-----+-----+
| Kịch bản 3: Kích hoạt Kiểm quyền Harness (Permission Guard) | ReAct Agent | WAIT_APPROVAL | 4 | 0.00s | - | CHẶN AN TOÀN |
+-----+-----+-----+-----+-----+-----+
| | Plan-then-Execute | WAIT_APPROVAL | 4 | 0.00s | - | CHẶN AN TOÀN |
+-----+-----+-----+-----+-----+-----+
| | Hybrid (Lai) | WAIT_APPROVAL | 6 | 0.00s | - | CHẶN AN TOÀN |
+-----+-----+-----+-----+-----+-----+
| ----- | ----- | ----- | ----- | ----- | ----- | ----- |
+-----+-----+-----+-----+-----+-----+

[CHÚ THÍCH]:
- 'ĐẠT (PASS)': Vượt qua tiêu chí kiểm chứng bằng code độc lập của CompletionSensor.
- 'CHẶN AN TOÀN': Lớp PermissionGuard đã ngắt luồng thành công trước khi gọi tool nhạy cảm book_ticket.
- 'Số vòng (Turns)': Số lượt tương tác Tool & suy luận của Agent.

Process finished with exit code 0
```

Hình 3: Bảng tổng hợp so sánh thực nghiệm 3 mẫu Agent qua 3 kịch bản từ benchmark.py

4) Nhận xét và phân tích chuyên sâu:

❖ Khả năng thích ứng với môi trường biến động:

- ReAct Agent: Thể hiện tính linh hoạt cao nhất. Ngay khi nhận được quan sát chuyến bay QH118 bị hết ghế, Agent lập tức thích ứng và đổi hướng sang kiểm tra chuyến thay thế VN122 chỉ sau 1 vòng lặp bổ sung (tổng 5 vòng).

- Plan-then-Execute: Nhờ được trang bị cơ chế LLM Dynamic Re-planner, Agent không bị gãy kế hoạch khi dữ liệu môi trường thay đổi. Re-planner nhận lại lý do thất bại và điều chỉnh chính xác các bước còn lại để hoàn tất đặt vé thành công.
- Hybrid (Lai): Kiến trúc phân tầng phát huy tối đa ưu điểm khi sự cố hết ghế được khoanh vùng và xử lý triệt để ngay bên trong Chặng 2 (xác minh ghế) mà không làm xáo trộn cấu trúc của Chặng 1 hay Chặng 3.

❖ Vai trò sống còn của Harness Code:

- Bảo đảm an toàn tài chính (PermissionGuard): Ở Kịch bản 3, cả 3 mẫu Agent đều bị chặn đứng an toàn trước khi hành động ghi nhận trừ tiền book_ticket diễn ra. Điều này khẳng định Agent không thể tự ý phát sinh chi phí nếu chưa có sự xác nhận của người dùng.
- Độ tin cậy của tiêu chí hoàn thành (CompletionSensor): 100% các lượt hoàn tất đều vượt qua kiểm tra đối chiếu trực tiếp trong Mock DB. Điều này loại bỏ hoàn toàn rủi ro Model sinh câu trả lời ảo nhưng không có vé thực tế.

5) KẾT LUẬN

- Quá trình thực nghiệm đã chứng minh rõ nét triết lý: Xây dựng một AI Agent ứng dụng trong thực tế không chỉ dừng lại ở việc kết nối Prompt với LLM, mà nằm ở việc thiết kế Harness Code vững chắc bao bọc xung quanh.
- Mô hình ngôn ngữ lớn (Model) đóng vai trò là "bộ não" suy luận linh hoạt, trong khi Harness đóng vai trò là "bộ khung xương" bảo đảm tính chính xác, an toàn dữ liệu và kiểm soát ngân sách vận hành.
- Việc lựa chọn mẫu thiết kế phụ thuộc vào bài toán: ReAct phù hợp với tác vụ nhanh, linh hoạt còn Plan-then-Execute tối ưu cho các quy trình tuần tự minh bạch và Hybrid là giải pháp toàn diện cho các hệ thống phức tạp đòi hỏi tính phân tầng và cô lập lỗi cao.